The Two Faces of Abstraction Regret: Control-Flow and Memory-Layout Limits of GPU DSLs on Irregular Automata

Alessandro Potenza
gpufsm project
ap.alessandro.potenza@gmail.com

***Abstract*—GPU domain-specific languages (DSLs) such as OpenAI Triton deliver near-CUDA performance at far lower effort on *regular* tensor algebra. We ask what that abstraction costs on *irregular* workloads and answer it with a metric we call *abstraction regret*: the performance a DSL forecloses—with the algorithm held fixed—because it cannot express the memory layout or control flow a workload needs. We decompose regret along these two capability axes and instantiate it on finite automata across the paradigm axis CUDA and NVIDIA Warp (thread-SIMT) versus Triton and its low-level Gluon frontend (tile-SPMD). Automata expose two complementary faces: an NFA active-set traversal that is *control-flow bound*, and a DFA dense-table walk that is *memory bound* (its throughput halves as the table crosses L2). On both faces the regret is large for the tile-SPMD DSLs and small for the thread-SIMT ones—Triton pays 5–$13\times$ versus CUDA across the two faces, while Warp, an equally high-level *Python* DSL, matches or beats hand CUDA on the NFA (0.6–$0.9\times$) and stays within $\sim 2\times$ on the DFA. So regret is set by the execution *paradigm*, not by how high-level the DSL looks. We make the attribution falsifiable with the Triton↔Gluon controlled pair (identical MLIR compiler stack; Gluon only adds explicit layout/shared-memory control): Gluon *still* cannot express the kernel, so the binding constraint is the paradigm, not tuning or layout. A two-parameter cost model corroborates the regret (predictive for the thread model; holdout 2.7%) and we name the missing IR primitives (scalar gather in a tile, register-resident bitset, data-dependent loop). Along the way we build a portable work-efficient automata engine ($\approx 330\times$–$10^4\times$ over a faithful full scan, 15–170 Gbps, validated bit-for-bit against a CPU oracle on six real ANMLZoo automata up to 48k states).**

I. Introduction

Irregular workloads—graph traversal, sparse algebra, automata—are dominated by data layout and data-dependent control flow rather than arithmetic. GPU DSLs raise the abstraction level by hiding exactly those concerns (thread indexing, memory placement, control flow), which is why they excel on dense tensor kernels. This paper measures what that hiding costs on NFA processing, a canonical irregular workload (deep-packet inspection, regular-expression matching, bioinformatics).

Contributions. (A) *Abstraction regret, operationalized*: a named framing decomposed along two capability axes (control-flow vs memory-layout) and instantiated on the *two faces* of automata—the control-flow-bound NFA and the memory-bound DFA; a two-parameter cost model (§III; predictive for the thread model, holdout-validated) and a constant-algorithm factorial ablation (§VI); and—the move that makes the attribution falsifiable rather than rhetorical—a controlled Triton↔Gluon pair (same MLIR stack, Gluon only *adds* layout control) plus a capability→cost table naming the missing IR primitive, attributing the DSL gap to the *execution paradigm*, distinct from generic performance portability and from autotuning. **(B)** *A work-efficient portable NFA engine*: an active-set/worklist bit-packed kernel (§IV) that removes the $O(n^2)$ compute wall and reaches 15–170 Gbps. **(C)** *A reproducible artifact*: one registry-based framework, a CPU reference oracle, median+CI95 sweeps, and figures regenerated only from versioned CSVs.

II. Background

NFA simulation maintains an active-state set and, per input symbol, applies the transition relation and an epsilon-closure. GPU engines since iNFAnt [1] store transitions in symbol-indexed / CSR form and represent the active set as a bit-vector. The modern frontier—AsyncAP [2], ngAP [3] (non-blocking + memoization + privatization), HybridSA [4] (bit-parallel), BitGen [5] (Parabix bitstreams)—wins by reorganizing or reducing irregular memory traffic, yet each frames its contribution as a new algorithm and bundles the memory effects in. Hyperscan [6] is the CPU baseline; ANMLZoo [7] and AutomataZoo are the standard suites. A *DFA* trades the NFA's parallel active set for a single dense transition table indexed once per input byte; it is the deterministic, *memory-bound* dual of the NFA and the second face we study. Crucially, every GPU automata engine above is CUDA-only: **no prior work compares modern GPU kernel DSLs (Triton/Gluon vs CUDA vs Warp) on an irregular workload**—existing DSL benchmarks (KernelBench, TritonBench) are dense-tensor-only, and the one irregular suite (ParEval) excludes these DSLs. We close that gap and frame the resulting gap as an expressibility cost.

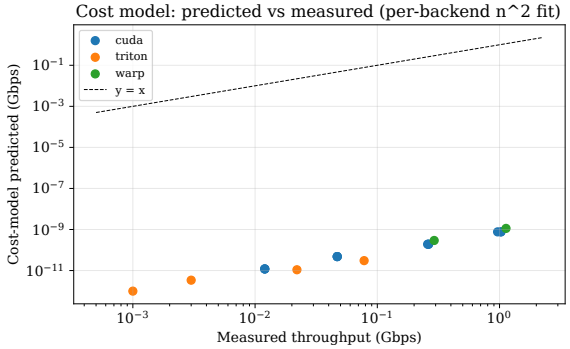

Fig. 1. Cost-model predicted vs measured throughput (per-backend n^2 fit); points near $y{=}x$ indicate the two-parameter model captures the regime.

III. Abstraction Regret and the Cost Model

We model per-symbol time as a roofline-style sum:

$$t_{\text{sym}} = \frac{\text{traffic}_{\text{bytes/sym}}}{\text{bandwidth}} + n^2 \cdot c_2, \qquad (1)$$

where n is the state count. The compute term is *quadratic* because the faithful kernel performs an $O(n)$ transition scan and an $O(n^2)$ epsilon-closure (n convergence passes $\times n$ states) per symbol. The two constants are fitted per backend from measured throughput; the ratio of c_2 between two DSLs, at constant algorithm, corroborates the abstraction regret. A holdout test (fit on $n \leq 128$, predict $n{=}256$) shows the model is *predictive for the thread-model backend* (CUDA 2.7% error, leave-one-out c_2 stable to $1.03\times$) but *not for the tile/SPMD backend* (Triton 45% error, c_2 swings $2.5\times$): Triton's large fixed launch overhead at small n is misattributed to the n^2 term (`scripts/validate_costmodel.py`). We therefore take the *directly measured* throughput ratio as the primary regret metric (Triton 6–8×, robust) and the fitted-c_2 ratio ($10.1\times$) as corroborating, not load-bearing (Fig. 1).

IV. Implementation

A single registry maps a backend/technique to an executor. The NFA is stored once in CSR and consumed identically by every backend, so comparisons are apples-to-apples. Correctness is gated against a CPU reference oracle (latch-first-match) and its bit-packed executable spec. **CUDA**: `dense` (int8/state), `bitpacked` (register-resident 64-bit words), `multistream` (one thread/string), `multistream_shared` (CSR staged in shared memory—a layout Triton cannot express), `multistream_async` (pinned + streamed overlap), and `worklist` (active-bit iteration via `__ffsll` + frontier eps-closure; O(active), no $O(n^2)$), `worklist_global` (global working set, dynamic word count, no 512-state cap — scales to ANMLZoo-sized automata; register residency costs ∼4–5× vs the capped kernel), and `worklist_warp` (*block-parallel*: one warp per string, the 32 lanes partition the state-words and scatter transitions via `atomicOr`—it spreads one string's loads across the warp, 3–9× faster than the single-thread global kernel on real ANMLZoo automata at a GPU-saturating batch, §VI), and `worklist_shared` (same scheme with the working set staged in dynamic *shared* memory, ≤1536 states—the working-set-layout ablation of the work-efficient kernel). **Triton**: `dense`, `bitpacked`, `multistream`, `worklist` (≤ 64 states, via `libdevice.ffs` + a data-dependent `while` loop). **Warp**: `multistream` (thread-SIMT Python). **Gluon** (Triton's experimental low-level frontend): `gl.load` always returns a layout-typed tensor (no scalar load), so the data-dependent CSR inner loop is inexpressible—a runnable probe (`scripts/gluon_probe.py`) reproduces the compiler error (*"Value argument cannot be block type"*) and exits non-zero if a future Gluon ever compiles it, making the claim falsifiable. **DFA (the memory-bound face):** a `dfa` kernel walks a dense *states* × 256 table, one random lookup per byte, implemented identically in CUDA, Triton and Warp (one program/thread per string); all match the DFA oracle.

TABLE I
Throughput (Gbps) by technique and NFA size (batched, RTX 4070). Dashes: > 64 states unsupported by the single-word Triton/Warp kernels.

states	CUDA worklist	CUDA full-scan	Triton full-scan	Triton worklist
32	170.2	0.513	0.071	24.4
64	163.8	0.131	0.021	25.2
128	47.6	0.023	0.003	–
256	31.7	0.006	0.001	–
500	15.5	0.0015	0.0002	–

V. Methodology

Throughput is measured on a fixed batch (2048×256 B) as total input bits per batch kernel time, reported as median + percentile-bootstrap 95% CI over 9 runs (GPU timings are non-Gaussian [8]), kernel and transfer time separated. Every configuration is verified bit-identical to the oracle on examples and randomized fuzz/stress NFAs (up to 500 states). Data and library versions are captured per CSV row. Hardware: NVIDIA RTX 4070 (sm_89, 46 SMs, 12 GB); software: CUDA toolkit 13.x, Triton 3.5.1, Warp 1.14, PyTorch 2.9 (cu128). Each datum is the median of 9 batch-kernel times; CSR transitions are read-only and shared across the batch. We additionally validate on *six real* ANMLZoo automata spanning six families: Levenshtein (2787 states), Hamming (11349, 2.1M transitions), Brill (42661, 4.4M), Fermi (40786, particle-physics regexes), RandomForest (33223, 6.27M transitions—an ML decision-forest, our densest), and CoreRings (48005, synthetic ring—our largest state count); all loaded from pinned public ANML with correct `all-input`/`start-of-data` semantics. `worklist_global` matches the reference oracle bit-for-bit on every input, confirming correctness at production scale (up to 48k states, 6.3M transitions).

VI. Results

Table I reports throughput; Table II the Nsight counters.

The faithful kernel is compute-bound; memory layout is irrelevant there. `multistream`, `multistream_shared` (modeled CSR traffic = 0), and `multistream_async` tie

TABLE II
NSIGHT COMPUTE COUNTERS (SINGLE LAUNCH). FULL-SCAN: SM THROUGHPUT ≫ DRAM ⇒ COMPUTE-BOUND; SHARED-CSR IS IDENTICAL BUT FOR L2 HIT RATE.

kernel	SM%	DRAM%	L2 hit%	occ%
`multistream` (full-scan)	19.44	0.01	79.2	16.7
`multistream_shared`	19.60	0.01	93.0	16.7
`worklist` (16384 str)	5.43	6.30	95.6	23.4

to within the bootstrap CI at every size (Fig. 4), and throughput scales as $1/n^2$ (Fig. 2). Nsight Compute counters confirm this at the hardware level: the full-scan kernel sustains ≈19% of peak SM throughput but only **0.01%** of peak DRAM throughput, and `multistream_shared` shows identical SM%, DRAM% and occupancy—only the L2 hit rate rises (79→93%), without moving runtime. The memory axes cannot help until the algorithm is work-efficient.

The work-efficient kernel unlocks the regime. The `worklist` kernel is ≈330×–10^4× faster than full-scan (the speedup grows with n: 332× at 32 states, ≈ 10^4× at 500; Fig. 3) and reaches 15–170 Gbps, moving the workload toward memory-bound where the memory techniques become load-bearing.

Abstraction regret is the execution paradigm. On the same full-scan kernel the directly-measured throughput ratio vs CUDA is Triton 6–8× and Warp 0.9× (Warp is *faster*; Fig. 5); the two-parameter cost-model fit corroborates this, isolating the per-DSL compute constant at Triton 10.1×, CUDA 1.0×, Warp 0.63×. Two equally high-level Python DSLs sit at opposite extremes: Triton's tile/SPMD paradigm strains to express per-state data-dependent control flow, Gluon cannot express it at all, while Warp's thread-SIMT model expresses it naturally and its codegen beats hand-written CUDA.

Expressibility ≠ efficiency. Triton *can* express the work-efficient worklist (unlike Gluon), yet still pays ≈ 6.5× vs CUDA there (CUDA 164 Gbps vs Triton 25 Gbps at ≤ 64 states)—essentially the same regret as the 6–8× on the full scan. Even expressing the right algorithm, the tile/SPMD model imposes a large constant penalty on scalar, data-dependent automata work.

The second face: DFA is memory-bound, and the regret persists. The DFA dense-table walk is the memory-bound dual of the NFA. A fine table-size sweep, median over 3 random DFAs per size (Fig. 6), makes the signature explicit: CUDA throughput rises to a peak *exactly* at the L2 capacity (364 Gbps at the 6 MB table) and then falls 2.2× monotonically to a DRAM-bound plateau (∼163 Gbps) once the table far exceeds L2. Warp tracks the same shape at about half (177 → 108 Gbps). **Triton, by contrast, is flat at** 29–32 **Gbps across the entire 1–100 MB range**—it never enters the memory-bound regime because its tile/SPMD codegen bottlenecks the scalar gather first (DFA regret 5–13×, largest where CUDA peaks at L2). So on *both* faces—control-flow-bound (NFA) and memory-bound (DFA)—the regret tracks the execution paradigm, not the workload's bottleneck: it is an intrinsic property of the DSL, not of where the kernel happens to be limited.

TABLE III
CAPABILITY → COST. ✓ EXPRESSIBLE, ∼ ONLY AS A STRAINED SINGLE PROGRAM, × INEXPRESSIBLE. REGRET IS THROUGHPUT VS CUDA ON EACH FACE.

Capability (paradigm)	CUDA thread-SIMT	Warp thread-SIMT	Triton tile-SPMD	Gluon tile-SPMD
Scalar element load	✓	✓	∼	×
Data-dependent loop	✓	✓	∼	∼
Register-resident bitset	✓	✓	×	×
Explicit shared-mem layout	✓	×	×	✓
NFA regret (control-flow)	1×	0.6–0.9×	6–10×	n/a (×)
DFA regret (memory)	1×	1.5–2.1×	5–13×	—

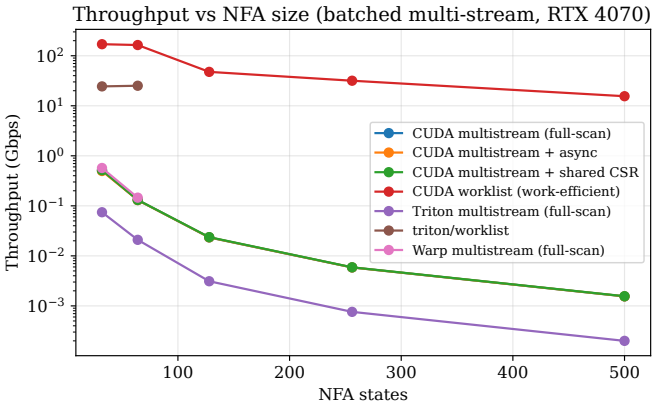

Fig. 2. Throughput vs NFA size (log scale). Worklist dominates; full-scan techniques collapse as $1/n^2$.

Capability → cost. Table III maps the capabilities each kernel needs to whether a DSL expresses them and the resulting regret. The thread-SIMT DSLs (CUDA, Warp) express all of scalar load, data-dependent loops, register-resident bitsets and explicit shared-memory layout; the tile-SPMD DSLs cannot express a scalar element load at all (Gluon) or only via a strained single program (Triton), which is exactly what the regret measures.

VII. RELATED WORK

Naming. We deliberately invert "abstraction *without* regret" (LMS [12]): staging removes call/dispatch overhead, but cannot manufacture a layout or control-flow pattern the surface abstraction forbids—which is precisely the residual we measure. **Closest quantitative prior, Hexcute** [13] ablates the Triton↔CUDA gap into layout-synthesis vs dataflow, but as a *compiler* on *dense tensor* kernels; we contribute a *metric* decomposed by *capability* (control-flow vs memory), on *irregular* automata, with an expressibility (not autotuning) framing and a falsifiable Triton↔Gluon control. **Tawa** [14] (warp specialization) and **Descend** [15] argue *qualitatively* that a DSL's execution model forecloses patterns; we quantify it cross-DSL. **Performance portability** [9] decomposes efficiency across a *hardware set*; we decompose across the *expressible-capability*

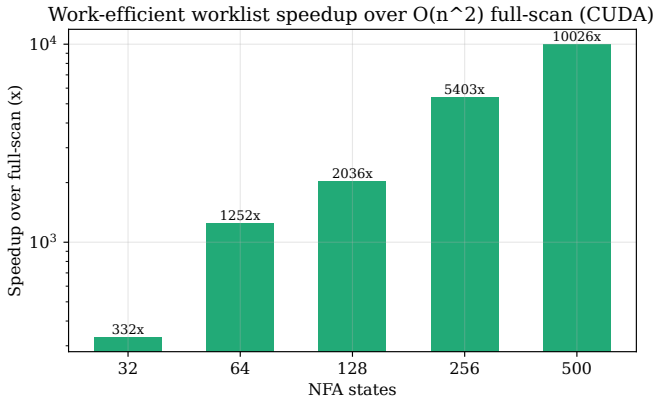

Fig. 3. Work-efficient worklist speedup over $O(n^2)$ full-scan (CUDA), growing with state count.

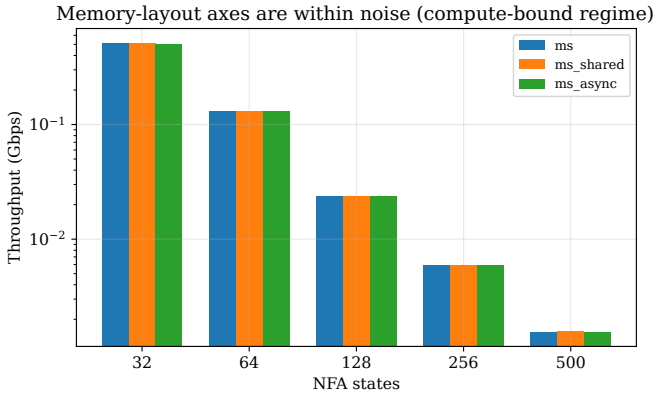

Fig. 4. `multistream` vs `_shared` vs `_async`: within noise at every size—compute-bound regime, memory layout irrelevant.

axis at fixed hardware. The Halide [10] lineage established abstraction constrains the schedule space; SpMV format selection and Gunrock [11] that layout dominates irregular GPU performance—we add the DSL-expressibility axis. We rebut the autotuning counter-thesis: our gap is expressibility, not tuning—Gluon, the explicit-control sibling on the same compiler stack, still cannot express the kernel. The same answers the recent LLM kernel auto-generation/tuning line (e.g. AutoTriton, TritonForge, 2026): automatically searching the Triton schedule space cannot manufacture an IR primitive (scalar gather in a tile) the paradigm lacks, so it cannot close a regret that is expressibility, not effort. SOTA automata engines (ngAP, HybridSA, BitGen) are CUDA-only baselines; our contribution is the metric + cost model + the first multi-DSL expressibility study of an irregular workload. Table IV positions us against them: each reports a speedup over *its own* baseline and hardware (not comparable in absolute Gbps), and each is a new *algorithm* on CUDA, whereas our axis is orthogonal—algorithm fixed, measuring what each DSL expresses. We therefore *position*, not benchmark, against them; matching ngAP-class absolute throughput is explicit future work (§IX).

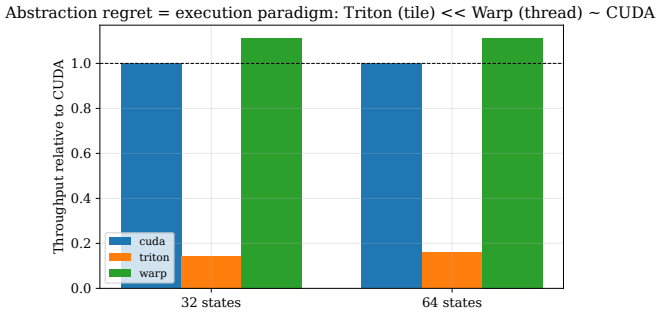

Fig. 5. Throughput relative to CUDA: Triton (tile) $\ll$ Warp (thread) $\approx$ CUDA. Regret is the execution paradigm, not abstraction height.

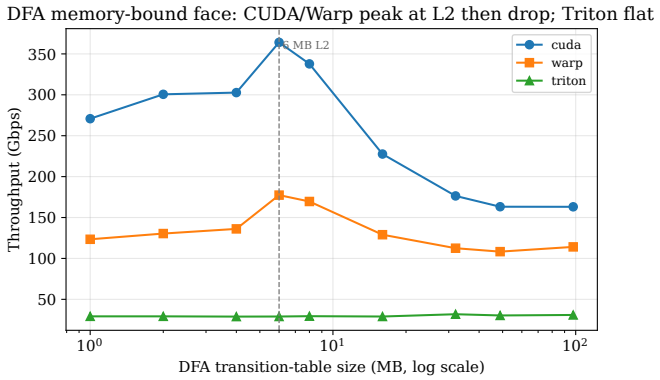

Fig. 6. DFA (memory-bound face), throughput vs table size (log x), median over 3 random DFAs/size: CUDA/Warp peak at the 6 MB L2 then drop $\sim$2.2$\times$ to a DRAM plateau; Triton stays flat (29–32 Gbps)—scalar-gather-bound, never reaching the memory regime.

VIII. Threats to Validity

Construct. Most throughput points use randomly generated NFAs, which may not represent production automata; we mitigate this by also validating on six real ANMLZoo automata spanning six families (2.8k–48k states, up to 6.3M transitions), where the GPU output matches the reference oracle bit-for-bit. *Internal.* Every backend/technique is checked against a single CPU oracle (latch-first-match) on examples and randomized fuzz/stress NFAs; timings use median + bootstrap CI on a fixed batch with warmup, separating kernel from transfer. *Implementation-effort asymmetry* (the key risk for a DSL comparison): the Triton 6–15$\times$ gap could merely reflect a weaker Triton implementation rather than an inherent limit. Two facts argue against this: (i) the kernels are structurally mirrored from the same CSR spec, and (ii) *Warp*—an equally high-level Python DSL written with comparable effort—matches or beats hand CUDA, so "high-level $\Rightarrow$ slow" is not the explanation; the tile/SPMD execution model is. *External.* Results are from a single GPU (RTX 4070); the cost-model constants are per-backend fits that may differ across architectures, so cross-architecture generality is future work.

TABLE IV
POSITIONING VS SOTA GPU AUTOMATA ENGINES. "REPORTED" IS EACH PAPER'S OWN SPEEDUP OVER ITS OWN BASELINE/HARDWARE—*not* COMPARABLE IN ABSOLUTE GBPS. ALL ARE CUDA-ONLY NEW ALGORITHMS; OUR AXIS (DSL EXPRESSIBILITY, FIXED ALGORITHM) IS ORTHOGONAL.

System (venue)	Mechanism	Reported (own basis)
iNFAnt (CCR'10)	symbol-indexed CSR, bit-vector	first GPU NFA
AsyncAP (SIGMETRICS'23)	input-symbol async parallelism	2.4–58×
ngAP (ASPLOS'24)	non-blocking + memoization	7.9× avg
HybridSA (OOPSLA'24)	bit-parallel + CPU/GPU split	bit-parallel
BitGen (MICRO'25)	Parabix bitstream fusion	19.5× vs GPU
This work	**DSL-regret metric, 4 DSLs**	**orthogonal**

IX. LIMITATIONS AND FUTURE WORK

Generality across architectures. Results are from one GPU (RTX 4070, sm_89, 6 MB L2). The qualitative claims should be architecture-independent—the capability table is a property of the DSL compilers, not the silicon, and the Triton↔Gluon control holds at compile time—but two quantities are L2- and SM-count-dependent and are the planned camera-ready run on a second architecture (e.g. an A100/H100, with ≥40 MB L2): (i) the DFA memory-bound knee, which should shift to a larger table size on a bigger L2 (a clean, falsifiable prediction of the memory-bound reading); and (ii) the absolute regret factors, since the per-backend cost-model constants are fits that may rescale. The entire sweep regenerates from one command, so the second-GPU run is a re-run, not a re-implementation. *Kernel.* The single-thread worklist under-utilizes the GPU on large automata (one string's many state-words processed serially; Nsight: 17% occupancy). Our `worklist_warp` kernel (one warp/string, 32 lanes partitioning the words) addresses this. The speedup is *batch- and density-dependent*: at a GPU-saturating batch (4096 strings) it is 3–9× faster on real ANMLZoo automata and ∼12× on dense synthetic NFAs; at small batch, where the single-thread kernel cannot fill the GPU, the gap is far larger (up to ∼180×). The warp kernel reaches its throughput plateau at a much smaller batch than the single-thread kernel (CSVs in `paper/data/`), so it is the right choice for few-stream / low-latency use. We further tested *shared-memory* working-set privatization (`worklist_shared`): it only ties `worklist_warp` (0.99–1.10×, ≤1536 states) — i.e. once the kernel is work-efficient the working-set *layout* is no longer the bottleneck (mirroring the compute-bound `multistream_shared` result). Nsight confirms the cause: `worklist_warp` fixes the single-thread kernel's occupancy (17 → 57%) but is *latency-bound*, not memory-bound—DRAM ≤2.25% and L2 hit ≥97.6% even for brill's 17 MB CSR (far above the 6 MB L2), since all strings share the CSR and only a hot row subset is touched (`paper/data/nsight_rtx4070.csv`). We also tested the natural work-reduction fix—a *compacted active-ID worklist* (O(active) instead of the O(words) bitmap scan)—but it barely beats the bitmap kernel (0.8–1.5×, `docs/KERNEL_EXPERIMENTS.md`): skipping an empty word is already cheap and the bitmap's coalesced access offsets compaction's scattered loads and dedup overhead, so word-scanning was never the bottleneck. The remaining gap to SOTA absolute throughput (ngAP-class) is therefore *algorithmic redundancy across strings/symbols*—memoization, non-blocking multi-symbol processing—not memory residency or worklist representation. That is the next step.

X. CONCLUSION

For irregular automata the GPU-DSL promise breaks along a measurable axis we call abstraction regret, governed by expressibility—of memory layout and, more sharply here, of data-dependent control flow—not by abstraction height. The memory-organization thesis holds, but only once the algorithm is work-efficient enough to be memory-bound, which our worklist engine achieves.

ARTIFACT AVAILABILITY

All code, the CPU reference oracle, the versioned result CSVs, and the figure generator are in the project repository. Every figure and table regenerates from the committed CSVs via a single command (`python paper/figures.py`); the CPU correctness suite runs without a GPU (`pytest -m "not gpu"`), and the GPU kernels and the Gluon falsifiability probe run on any CUDA device. A versioned snapshot with a Zenodo DOI will be minted at release for artifact evaluation. The claim-to-command map is in `docs/REPRODUCIBILITY.md` and a SIGPLAN-style artifact appendix (checklist, install, claims→commands→expected) in `docs/ARTIFACT_APPENDIX.md`.

REFERENCES

[1] N. Cascarano *et al.*, "iNFAnt: NFA pattern matching on GPGPU devices," *ACM SIGCOMM CCR*, 2010.

[2] H. Liu, S. Pai, A. Jog, "Asynchronous Automata Processing on GPUs," *POMACS/SIGMETRICS*, 2023.

[3] T. Ge, T. Zhang, H. Liu, "ngAP: Non-blocking Large-scale Automata Processing on GPUs," *ASPLOS*, 2024.

[4] A. Le Glaunec, L. Kong, K. Mamouras, "HybridSA: GPU Acceleration of Multi-pattern Regex Matching using Bit Parallelism," *OOPSLA*, 2024.

[5] T. Ge, X. Chu, H. Liu, "Interleaved Bitstream Execution for Multi-Pattern Regex Matching on GPUs," *MICRO*, 2025.

[6] X. Wang *et al.*, "Hyperscan: A Fast Multi-pattern Regex Matcher for Modern CPUs," *USENIX NSDI*, 2019.

[7] J. Wadden *et al.*, "ANMLZoo: a benchmark suite for exploring bottlenecks in automata processing," *IISWC*, 2016.

[8] T. Hoefler, R. Belli, "Scientific Benchmarking of Parallel Computing Systems," *SC*, 2015.

[9] S. J. Pennycook, J. D. Sewall, V. W. Lee, "A Metric for Performance Portability," *PMBS@SC / FGCS*, 2016/2019.

[10] J. Ragan-Kelley *et al.*, "Halide: a language and compiler for optimizing parallelism, locality, and recomputation," *PLDI*, 2013.

[11] Y. Wang *et al.*, "Gunrock: A High-Performance Graph Processing Library on the GPU," *PPoPP*, 2016.

[12] T. Rompf, M. Odersky, "Lightweight Modular Staging: Abstraction Without Regret," *CACM* 55(6), 2012.

[13] X. Zhang *et al.*, "Hexcute: A Tile-based Programming Language with Automatic Layout and Task-Mapping Synthesis," arXiv:2504.16214, 2025 (preprint).

[14] Authors, "Tawa: Automatic Warp Specialization for GPU Tile Languages," *CGO*, 2026 (to appear; arXiv:2510.14719).

[15] B. Köpcke, S. Gorlatch, M. Steuwer, "Descend: A Safe GPU Systems Programming Language," *PLDI*, 2024.